

UNIVERSIDAD NACIONAL DE LA PLATA

FACULTAD DE INFORMÁTICA

Programación Distribuida y Tiempo Real (PDyTR)

Sistema de Visión Computacional Distribuido y Monitoreo Inteligente en el Borde

Documentación Técnica de Arquitectura, Virtualización con Vagrant, Aprovisionamiento y
Módulos de Procesamiento en el Borde

INFORME FINAL DE ENTREGA

AUTOR / DESARROLLADOR

Nicolas Ricciardi

AUTOR / DESARROLLADOR

Fabrizio Torrico

Fecha: Agosto de 2026 | Versión: 1.0 Final | Tecnologías: Java 21, OpenCV, Vagrant, VirtualBox, React + Vite

Índice General del Documento

1. Funcionamiento General del Sistema	Sección 1
1.1. Propósito y Paradigma de Edge Computing	1.1
1.2. Diagrama de Arquitectura Global	1.2
1.3. Componentes Principales	1.3
1.4. Máquina de Estados de los Nodos Edge	1.4
1.5. Protocolo de Comunicación de Red	1.5
2. Estrategia de Virtualización y Aprovisionamiento	Sección 2
2.1. Arquitectura de Red y Virtualización	2.1
2.2. Aprovisionamiento vs. Despliegue en Producción	2.2
3. Documentación del Vagrantfile y Dependencias	Sección 3
3.1. Configuración Base Global	3.1
3.2. Sección del Servidor Central (central_server)	3.2
3.3. Secciones de los Nodos de Borde (edge_node_1 y edge_node_2)	3.3
4. Virtualización y Módulos del Nodo de Borde (edgeNode)	Sección 4
4.1. Fuentes que Corren en los Edges	4.1
4.2. Instalación y Modalidades de Ejecución	4.2
5. Virtualización y Módulos del Servidor Central (centralServer)	Sección 5
5.1. Fuentes que Corren en el Servidor	5.1
5.2. Instalación y Ejecución del Servidor	5.2
6. Integración con el Cliente Web Dashboard (webClient)	Sección 6
7. Resumen Consolidado de Dependencias y Requisitos	Sección 7
8. Guía de Verificación y Puesta en Marcha	Sección 8

1. Funcionamiento General del Sistema

1.1. Propósito y Paradigma de Edge Computing

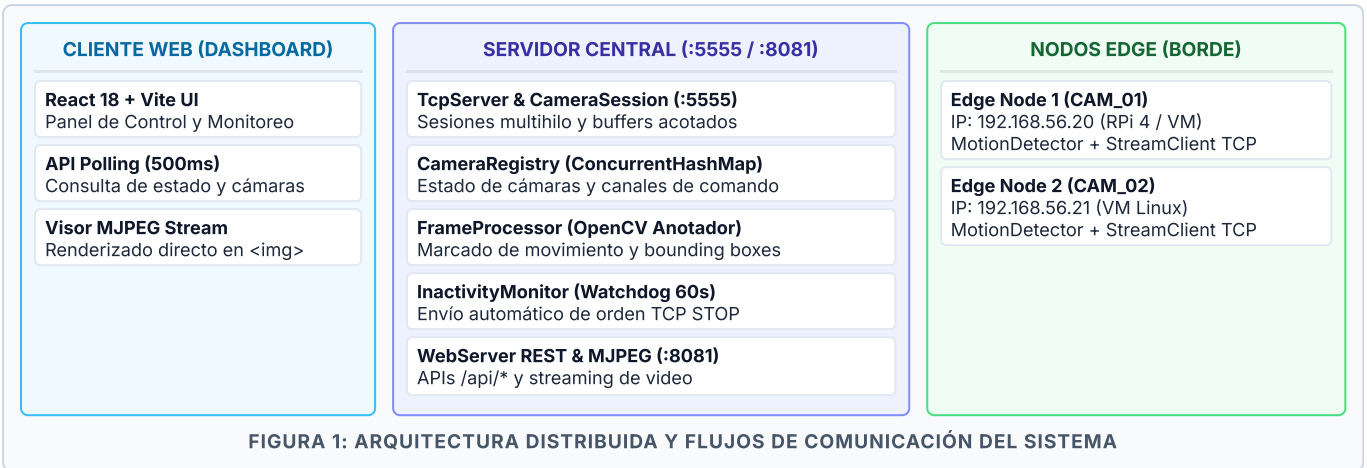
El sistema implementa una arquitectura distribuida de videovigilancia orientada a la **optimización del uso de ancho de banda en la red y del procesamiento central**, dando cumplimiento a los requerimientos del trabajo final de la cátedra de Programación Distribuida y Tiempo Real (PDyTR).

En concordancia con las definiciones metodológicas fijadas en las reuniones de seguimiento de la bitácora, se mantiene estricta consistencia en la terminología empleada: los términos *"Edge"* y *"Nodo de Borde"* representan conceptualmente el cómputo en el extremo de la red (implementado tanto sobre una **Raspberry Pi 4B** física como sobre máquinas virtuales Linux), mientras que *"Central Server"* y *"Back End"* corresponden al servidor centralizador.

En los esquemas convencionales de CCTV, todas las cámaras transmiten de forma ininterrumpida flujos de video completos hacia un servidor central, lo que satura los canales de comunicación y genera cuellos de botella de procesamiento. Para mitigar esta problemática, el presente proyecto aplica el paradigma de **Edge Computing (Computación en el Borde)**:

- **Definición de Video "de Interés"**: Siguiendo el alcance delimitado por la cátedra, se define video de interés estrictamente como **video con identificación de movimiento**. Los nodos de borde transmiten hacia el servidor central *únicamente* cuando se discrimina movimiento local en la escena vigilada.
- **Procesamiento en el Borde**: Cada nodo de borde analiza localmente las imágenes capturadas utilizando técnicas de visión por computadora livianas implementadas con OpenCV.
- **Silencio de Red (Ahorro de Recursos)**: Mientras no se registre actividad en la escena, los nodos no transmiten video por la red, permaneciendo en estado de análisis local silencioso.
- **Transmisión Conducida por Eventos**: Al detectarse movimiento relevante (o ante la petición explícita de un operador desde el cliente web), el nodo conmuta a modo de transmisión activa y envía los fotogramas comprimidos mediante un socket TCP hacia el Servidor Central.
- **Procesamiento Central y Distribución**: El Servidor Central recibe los fotogramas, aplica algoritmos de sustracción y marcado para delimitar visualmente el movimiento con recuadros verdes y leyendas de alerta, gestiona temporizadores de inactividad para ordenar el retorno al modo silencioso cuando cesa la actividad, y expone los flujos de video mediante MJPEG y APIs REST hacia el Dashboard web.
- **Topología Distribuida**: El sistema opera con al menos 2 nodos edge concurrentes (un nodo real sobre Raspberry Pi 4 y nodos simulados/virtualizados en PC mediante Vagrant).

1.2. Diagrama de Arquitectura Global

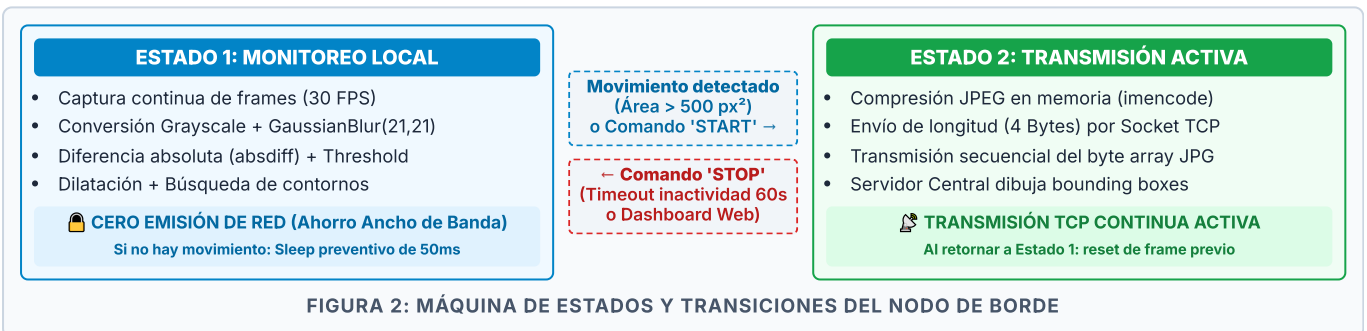


1.3. Componentes Principales

- Nodos de Borde (`edgeNode`):** Capturan video desde dispositivos de captura física (webcam USB vía V4L2) o archivos de video de prueba, ejecutan el algoritmo de detección de movimiento local en memoria y transmiten fotogramas únicamente bajo demanda o detección confirmada.
- Servidor Central (`centralServer`):** Orquestador multihilo que administra las conexiones TCP de las cámaras, desacopla la recepción mediante colas acotadas anti-lag (capacidad 5), anota fotogramas resaltando áreas en movimiento, supervisa la inactividad y sirve la API REST y los streams MJPEG.
- Cliente Web (`webClient`):** Single Page Application construida en React + Vite que proporciona un panel de monitoreo y control en tiempo real.

1.4. Máquina de Estados de los Nodos Edge

Cada nodo de borde implementa una máquina de estados finita estrictamente excluyente con dos modos de funcionamiento (establecidos junto a la cátedra el 02/02/26):



Justificación del Algoritmo en el Edge (Visión Clásica vs. YOLO): Durante la fase de prototipado se evaluaron modelos de redes neuronales (YOLOV5 Nano) sobre la CPU de la Raspberry Pi 4B. Las mediciones mostraron una degradación crítica del rendimiento a ~5 FPS y un consumo excesivo de CPU/temperatura. Siguiendo las directivas docentes, se consolidó el algoritmo clásico determinístico con OpenCV (diferencia de fotogramas sucesivos, desenfoque gaussiano y análisis de contornos), lo cual asegura una tasa fluida y estable de **~30 FPS** con mínimo consumo computacional.

1.5. Protocolo de Comunicación de Red

- Handshake Inicial:** El nodo Edge abre una conexión TCP hacia el puerto 5555 del Servidor Central y transmite como primer dato una cadena UTF-8 con su identificador único (`camId`). El servidor registra el socket asociado a dicha identidad e IP.
- Trama de Video Binaria:** En el Estado 2, cada fotograma se transmite estructurado en dos partes:
 - Encabezado de Tamaño:** Entero de 4 bytes (formato Big-Endian / `writeInt`) indicando la longitud en bytes del fotograma JPEG.
 - Carga Útil (Payload):** Arreglo de bytes crudos de la imagen JPEG (`write(imageBytes)`).
- Canal de Control Bidireccional sobre TCP:** Conforme a la reunión del 08/04/26, se descartó el uso de servidores HTTP en el edge para simplificar el nodo y ahorrar recursos. A través del mismo socket TCP ya establecido, el Servidor Central envía cadenas de comando UTF-8:
 - START** : Ordena al Edge conmutar inmediatamente al Estado 2 (Streaming).
 - STOP** : Ordena al Edge conmutar inmediatamente al Estado 1 (Monitoreo silencioso).
- API REST y Streaming MJPEG Web:**

- `GET /api/cameras`: Retorna un JSON con la lista de cámaras registradas, direcciones IP y estado booleano de streaming.
- `GET /api/start?id=CAM_ID` y `GET /api/stop?id=CAM_ID`: Control manual remoto desde el dashboard.
- `GET /stream?id=CAM_ID`: Flujo de video continuo con cabecera `multipart/x-mixed-replace; boundary=--BoundaryString` compatible con elementos estándar HTML.

2. Estrategia de Virtualización y Aprovisionamiento

2.1. Arquitectura de Red y Virtualización

Para garantizar el aislamiento de procesos y total reproducibilidad en la evaluación del sistema (según lo conversado en la bitácora respecto a no utilizar Docker sino máquinas virtuales completas), la solución se virtualiza utilizando **Vagrant** sobre el hipervisor **Oracle VirtualBox**. La topología está configurada con una Red Privada (Host-Only Network) bajo el rango `192.168.56.0/24`:

Máquina Virtual	Hostname	IP Privada	Servicios y Puertos	Recursos
<code>central_server</code>	<code>centralserver</code>	<code>192.168.56.10</code>	TCP :5555 (Video/Comandos) HTTP :8081 (Forwarded a Host:8081)	1 CPU, 1024 MB RAM
<code>edge_node_1</code>	<code>edgenode1</code>	<code>192.168.56.20</code>	Cliente TCP hacia <code>192.168.56.10:5555</code> Controlador USB habilitado	1 CPU, 1024 MB RAM
<code>edge_node_2</code>	<code>edgenode2</code>	<code>192.168.56.21</code>	Cliente TCP hacia <code>192.168.56.10:5555</code> Controlador USB habilitado	1 CPU, 1024 MB RAM

2.2. Aprovisionamiento vs. Despliegue en Producción

Nota Técnica sobre el Modelo de Entrega y Aprovisionamiento (Reunión del 08/04/26)

En la configuración de Vagrant de este proyecto, el directorio del repositorio en la máquina anfitriona se monta automáticamente en el punto `/vagrant` dentro de cada máquina virtual mediante carpetas compartidas.

Durante la etapa de **aprovisionamiento** (`provision shell`), se instalan los paquetes del sistema operativo base (Ubuntu 22.04 LTS), OpenJDK 21, Apache Maven y utilidades de video. Inmediatamente a continuación, el aprovisionamiento ingresa a la carpeta compartida correspondiente (`/vagrant/centralServer` o `/vagrant/edgeNode`) y compila el código fuente ejecutando `mvn clean compile`, generando asimismo los scripts ejecutables de inicio (`start_central.sh` y `start_edge.sh`).

Aclaración de Diseño: Se explicita que compilar el código fuente dentro del script de aprovisionamiento de una máquina virtual no representa un proceso de despliegue productivo estándar (donde se generarían artefactos inmutables precompilados, imágenes de contenedor o paquetes Debian versionados). Para los fines de esta entrega académica, este mecanismo garantiza máxima **transparencia, reproducibilidad y verificación directa** de la compilación y ejecución del código fuente sin requerir binarios pesados en el repositorio.

3. Documentación del Vagrantfile y Dependencias

El archivo de configuración `Vagrantfile` define de forma declarativa toda la infraestructura virtual y sus dependencias explícitas:

3.1. Configuración Base Global

Se utiliza la imagen oficial `ubuntu/jammy64` (Ubuntu Server 22.04 LTS de 64 bits), proveyendo un entorno Linux homogéneo y estable para la ejecución de Java 21 y la manipulación de dispositivos de video.

3.2. Sección del Servidor Central (`central_server`)

- **Red y Puertos:** IP estática privada `192.168.56.10` y redirección del puerto `8081` del huésped hacia el puerto `8081` del anfitrión para permitir el acceso del dashboard web en React.
- **Hardware Virtual:** 1 núcleo vCPU y 1024 MB de memoria RAM.
- **Dependencias Explícitas de Aprovisionamiento:** `openjdk-21-jdk` y `maven`.
- **Compilación y Script de Inicio:** Ejecuta `mvn clean compile` en `/vagrant/centralServer` y crea el script ejecutable `/home/vagrant/start_central.sh`.

3.3. Secciones de los Nodos de Borde (`edge_node_1` y `edge_node_2`)

- **Red:** IPs estáticas privadas `192.168.56.20` y `192.168.56.21`.
- **Controladores USB en VirtualBox:** Activa los flags `--usb on`, `--usbhci on` y `--usbxhci on` para permitir el passthrough de cámaras físicas USB 2.0 y 3.0 hacia la máquina virtual.

- **Hardware Virtual:** 1 núcleo vCPU y 1024 MB de memoria RAM por cada nodo.
- **Dependencias Explícitas de Aprovisionamiento:** `openjdk-21-jdk`, `maven` y `v4l-utils` (Video4Linux2).
- **Compilación y Script de Inicio Parametrizado:** Ejecuta `mvn clean compile` en `/vagrant/edgeNode` y genera `/home/vagrant/start_edge.sh` admitiendo parámetros para seleccionar el índice de cámara física (0, 1) o la ruta a un archivo de video simulado.

4. Virtualización y Módulos del Nodo de Borde (`edgeNode`)

4.1. Fuentes que Corren en los Edges

Los módulos de software que componen el nodo de borde se ubican en el paquete `org.alumnosinfo.tpdistribuido` dentro de la carpeta `edgeNode/`:

- **EdgeNode.java**: Orquestador principal del nodo. Carga dinámicamente las librerías nativas de OpenCV (`nu.pattern.OpenCV.loadLocally()`), procesa los argumentos de ejecución (IP del servidor, ID de cámara y fuente de video), gestiona la reconexión automática ante caídas de red, computa y reporta métricas periódicas de FPS, y coordina el bucle de captura y streaming.
- **EdgeStateManager.java**: Modela el estado operativo del nodo de borde de forma segura para hilos mediante variables marcadas con `volatile` (`streamingMode` y `resetPrevGray`), asegurando sincronización inmediata entre el hilo de comandos TCP y el hilo de captura de video.
- **MotionDetector.java**: Implementa el algoritmo de visión por computadora para detección de movimiento en el borde. Transforma la imagen a escala de grises, aplica desenfoque gaussiano de 21×21 , efectúa sustracción de fondo absoluta (`absdiff`), binariza por umbral y dilata la imagen. Confirma detección si el área de algún contorno excede los 500 píxeles cuadrados.
- **StreamClient.java**: Administra el socket TCP cliente. Ejecuta el apretón de manos inicial, mantiene un hilo escucha para comandos remotos (`START` / `STOP`), y en el modo de transmisión comprime los fotogramas a JPEG y los envía con enmarcado de longitud de 4 bytes en bloques sincronizados.
- **VideoSource.java**: Capa de abstracción para la captura de video que soporta indistintamente dispositivos físicos (V4L2 en Linux) y archivos de video pregrabados (`.mp4`). Para archivos de video, implementa rebobinado automático al final de la pista para simulación continua.
- **pom.xml**: Define las dependencias de Maven del nodo, incluyendo `org.opennnp:opencv:4.9.0-0` y el plugin `maven-shade-plugin` para empaquetado autónomo.
- **edgenode.service**: Archivo de configuración para registrar el nodo como un servicio administrado por **systemd** en dispositivos físicos independientes (como Raspberry Pi OS).

4.2. Instalación y Modalidades de Ejecución

- **Aprovisionamiento en la VM:** Realizado automáticamente al ejecutar `vagrant up`.
- **Ejecución con Cámara Física:** Habiendo capturado el dispositivo USB en VirtualBox, se ejecuta `./start_edge.sh 0` dentro de la VM.
- **Ejecución con Video Simulado:** Se pasa como argumento la ruta al archivo de video: `./start_edge.sh /vagrant/videos/prueba.mp4`.

5. Virtualización y Módulos del Servidor Central (`centralServer`)

5.1. Fuentes que Corren en el Servidor

Los módulos de software del Servidor Central se encuentran en la carpeta `centralServer/`:

- **CentralServer.java**: Punto de entrada del servidor. Carga librerías nativas de OpenCV, inicializa el `CameraRegistry`, inicia el `InactivityMonitor`, y levanta el servidor HTTP (:8081) y el servidor TCP (:5555) en hilos dedicados.
- **CameraRegistry.java**: Estructura de datos thread-safe (`ConcurrentHashMap`) que centraliza el estado de las cámaras: últimos fotogramas procesados, IPs, flujos de salida para comandos, y marcas temporales de último frame y último movimiento registrado.
- **TcpServer.java** y **CameraSession.java**: Servidor de sockets TCP multihilo. Por cada nodo conectado crea una sesión que encola los fotogramas entrantes en un búfer acotado (`LinkedBlockingQueue(5)`) descartando frames obsoletos ante saturación para eliminar la latencia acumulada (lag).
- **FrameProcessor.java**: Hilo de procesamiento de imagen que desencola frames, detecta contornos con movimiento, dibuja cuadros delimitadores verdes, estampa la leyenda `MOVIMIENTO DETECTADO` en texto rojo y actualiza el registro central con el fotograma anotado.
- **InactivityMonitor.java**: Tarea periódica programada cada 10 segundos que audita las cámaras activas. Si transcurren más de 60 segundos sin registrarse movimiento en la escena, emite automáticamente la orden TCP `STOP` para apagar la

transmisión del nodo.

- **Subpaquete Web (WebServer y Handlers):**

- `WebServer.java` : Servidor HTTP embebido en el puerto 8081 con pool de hilos en caché.
 - `BaseHttpHandler.java` : Manejador base con soporte para cabeceras CORS y resolución de solicitudes OPTIONS.
 - `CamerasApiHandler.java` : Endpoint `GET /api/cameras` que lista las cámaras y su estado en JSON.
 - `CommandApiHandler.java` : Endpoints `GET /api/start` y `GET /api/stop` para control remoto.
 - `StreamHandler.java` : Endpoint `GET /stream?id=CAM_ID` con streaming MJPEG `multipart/x-mixed-replace`.
 - `CameraDto.java` : Objeto de transferencia de datos para serialización JSON.
- `pom.xml` : Define la compilación con Java 21 y la dependencia de OpenCV.

5.2. Instalación y Ejecución del Servidor

El aprovisionamiento de Vagrant instala Java 21 y Maven, compila el proyecto y deja listo el script `/home/vagrant/start_central.sh`. Para iniciar el servidor dentro de la máquina virtual:

```
Comandos de Inicio del Servidor Central:  
vagrant ssh central_server  
./start_central.sh
```

6. Integración con el Cliente Web Dashboard (webClient)

El cliente web es una aplicación desarrollada en **React + Vite** ubicada en `webClient/`:

- **Monitoreo en Tiempo Real:** Consulta periódicamente `GET /api/cameras` para listar nodos conectados y su estado.
- **Reproducción MJPEG:** Incrusta el stream de video directamente en etiquetas ``.
- **Control Remoto:** Permite forzar el inicio o la detención de cualquier cámara mediante peticiones a `/api/start` y `/api/stop`.
- **Ejecución en Host:** Se instala y ejecuta en la máquina anfitriona con `npm install && npm run dev`, accediendo en `http://localhost:5173`.

7. Resumen Consolidado de Dependencias y Requisitos

Componente	Software / Herramienta	Versión	Función en el Sistema
Virtualización	Vagrant	≥ 2.3	Definición y aprovisionamiento automatizado de las VMs
Hipervisor	Oracle VirtualBox	≥ 6.1 / 7.0	Ejecución de VMs y emulación de controladores USB
Sistema Huésped	Ubuntu Server	22.04 LTS	Sistema operativo base en todas las máquinas virtuales
Lenguaje	OpenJDK	21	Compilación y ejecución de aplicaciones Java
Construcción	Apache Maven	≥ 3.8	Gestión de dependencias y ciclo de vida de compilación
Visión Artificial	OpenCV (OpenPnP)	4.9.0-0	Procesamiento de imágenes y algoritmos de detección
Captura Linux	v4l-utils / V4L2	Nativo	Control y captura de dispositivos de video en Linux
Frontend Web	React + Vite (Node.js)	Node ≥ 18	Dashboard interactivo de usuario y visualización

8. Guía de Verificación y Puesta en Marcha

8.1. Despliegue Automatizado con Vagrant (Recomendado)

1. Levantar la Infraestructura Virtual (en la raíz del proyecto):

```
vagrant up
```

2. Iniciar el Servidor Central (Terminal 1):

```
vagrant ssh central_server → ./start_central.sh
```

3. Iniciar el Edge Node 1 (Terminal 2):

```
vagrant ssh edge_node_1 → ./start_edge.sh 0 (o con ruta a video)
```

4. Iniciar el Edge Node 2 (Terminal 3):

```
vagrant ssh edge_node_2 → ./start_edge.sh 0 (o con ruta a video)
```

5. Iniciar el Dashboard Web en el Host (Terminal 4):

```
cd webClient → npm run dev → Abrir http://localhost:5173
```

8.2. Ejecución Nativa en Host (Sin Virtualización)

1. Servidor Central:

```
cd centralServer && mvn exec:java -Dexec.mainClass="org.alumnosinfo.tpdistribuido.CentralServer"
```

2. Edge Node 1:

```
cd edgeNode && mvn exec:java -Dexec.mainClass="org.alumnosinfo.tpdistribuido.EdgeNode" -Dexec.args="localhost CAM_01 0"
```

3. Edge Node 2:

```
cd edgeNode && mvn exec:java -Dexec.mainClass="org.alumnosinfo.tpdistribuido.EdgeNode" -Dexec.args="localhost CAM_02 1"
```

4. Web Client:

```
cd webClient && npm run dev
```